Predicting Developer Productivity and Code Quality in AI-Assisted Environments Using Random Forest Classification

TECHNICAL ANALYSIS REPORT

Prepared by	Muhammad Ali
Institution	National Textile University
Date	September 9, 2026

Abstract

The following comprehensive report builds on the initial study of AI-enabled developer productivity. By utilizing a data set with 500 workflow sessions, machine learning techniques (Random Forest Classification and Regression) were used to forecast both task success and code quality (bug count). The analysis graphically represents the trade-off between task completion and code quality. Most importantly, the models show that whereas coding hours ensure success, bad sleep and excessive AI use hours are key predictors for errors.

Objectives

- Train ML models to predict task success and defect rates.
- Evaluate models using Confusion Matrices and F1-scores to take into consideration class imbalance problems.
- Identify feature importance to distinguish factors affecting productivity from those causing defects.
- Visualize the results to get actionable insights for AI-assisted software engineering.

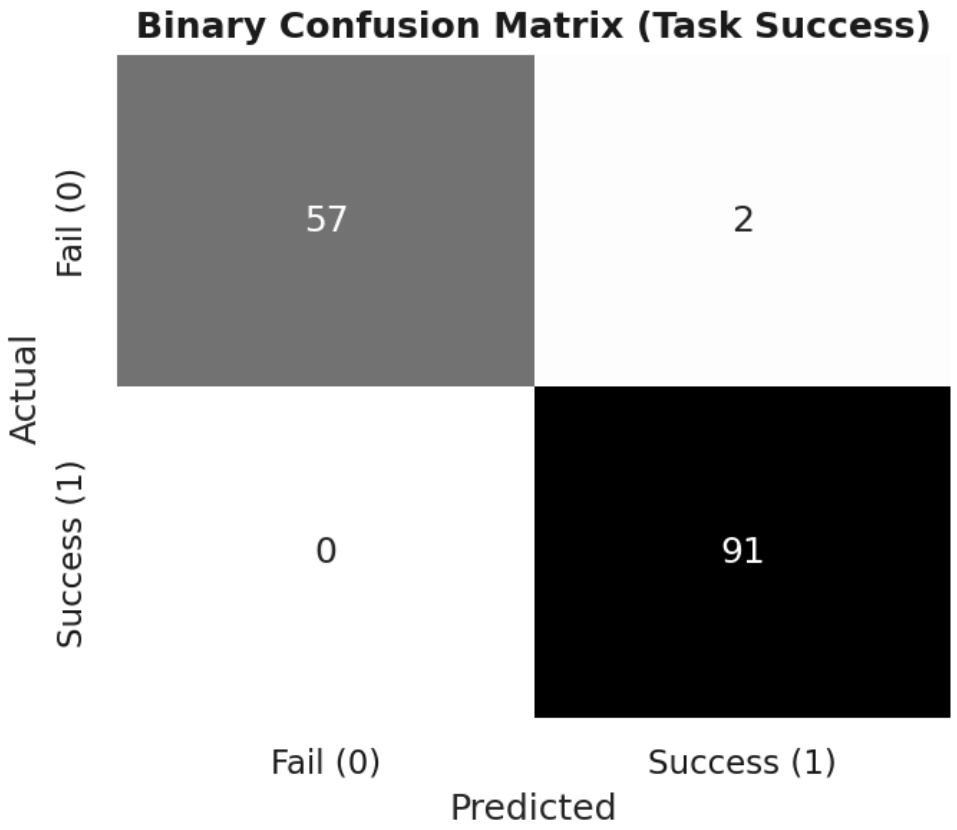
Methodology & Results

The data were stratifiedly split (70/30) and trained on the random forest algorithms (n_estimators = 100, class_weight

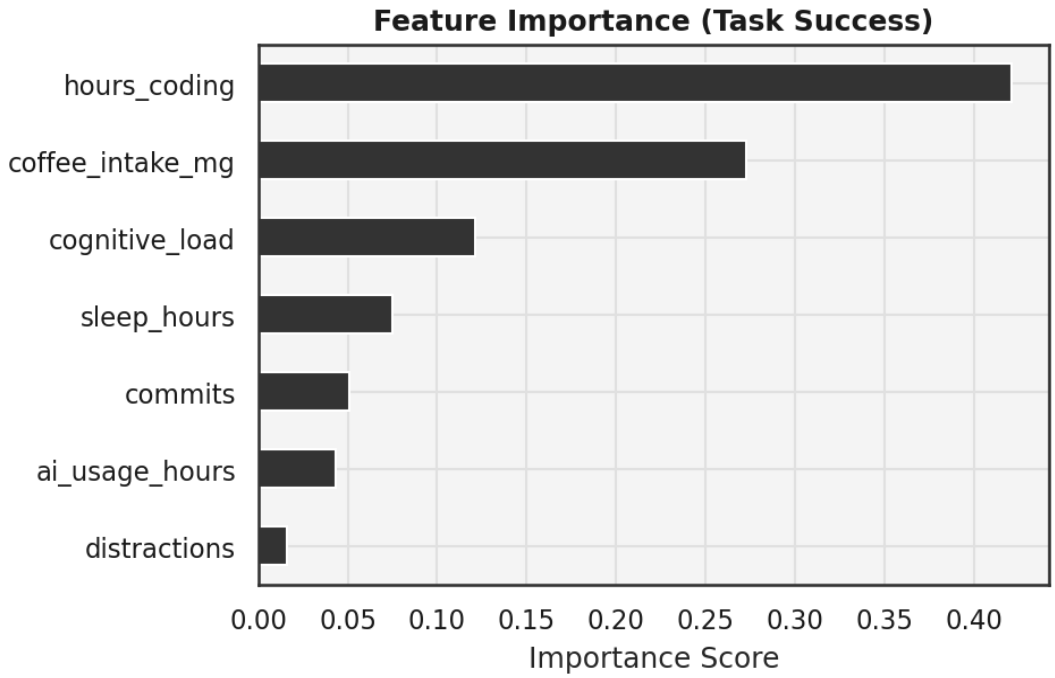
Binary Classification (Task Success)

The model produced an overall accuracy of 99%. The confusion matrix provided below clearly shows the distinct separability of task completion, whereas the feature importance graph indicates that 'hours_coding' is the most influential variable.

Confusion Matrix:



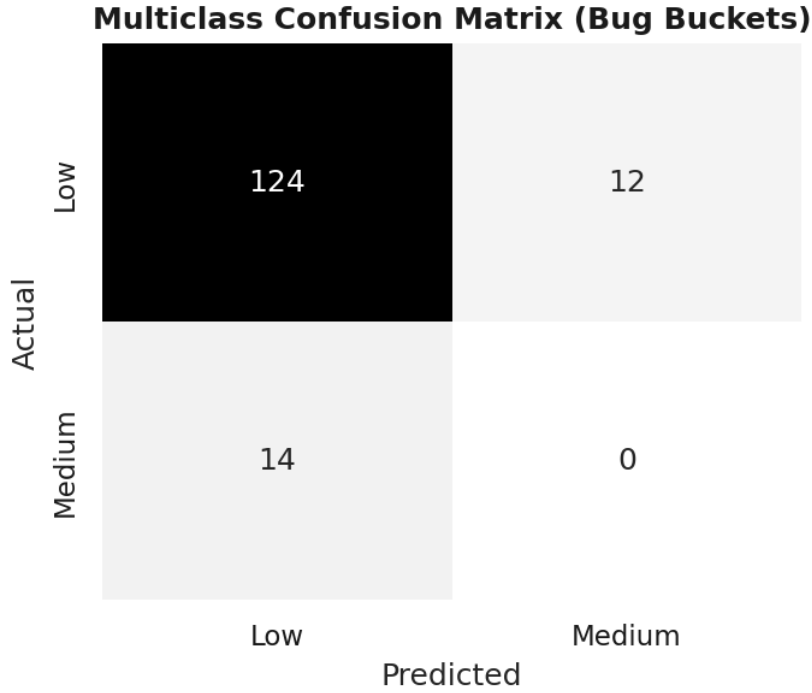
Feature Importance Graphs:





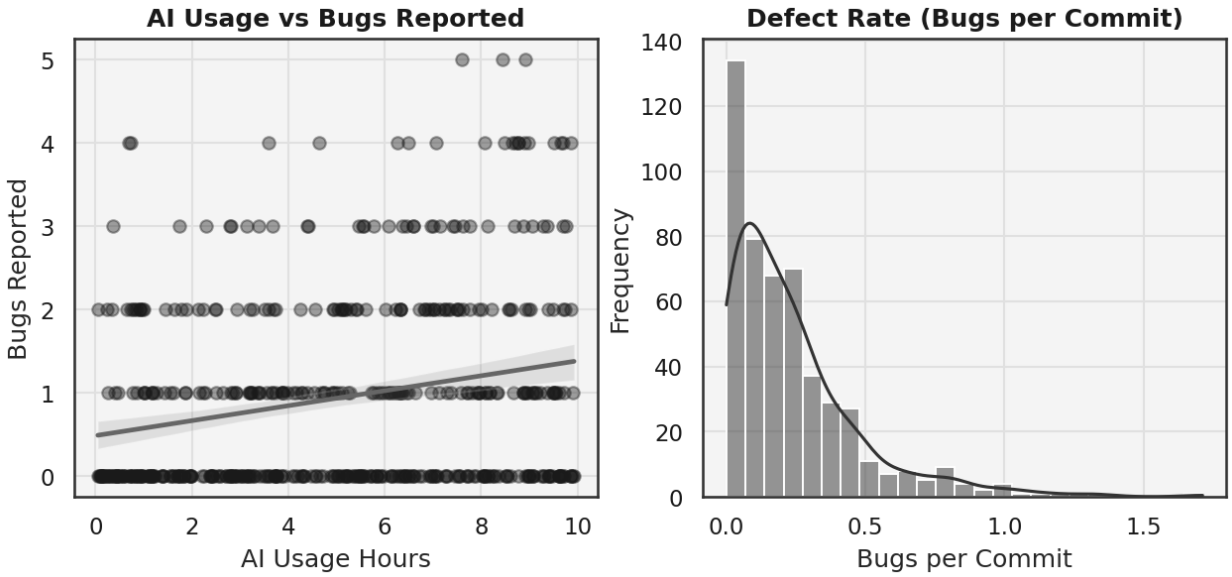
Multiclass Classification (Code Quality)

The classification of bug buckets (Low vs. Medium) was difficult because of data scarcity with respect to the minority category (Precision 0.00 for Medium bugs). It is true to life that anomaly detection poses several problems. But more importantly, the feature drivers got switched around ('sleep_hours', 'ai_usage_hours' trumped 'hours_coding').



Regression and Defect Rate Distributions

Regression analysis further confirmed that there is a positive relationship (+0.11) between the use of AI and number of bugs. The Defect Rate measure (bugs per commit) was created, which showed an average of 0.230 bugs per commit.



Conclusion: The AI Paradox

The visual and mathematical evidence indicates that there is an "AI Paradox" when it comes to modern software engineering. The software tools do manage to speed up the process of doing standard tasks, but they also end up contributing to increased numbers of defects unless controlled. The future phases of this project will focus on exploring SMOTE over-sampling and anomaly detection paradigms.